FUCAPE
BUSINESS SCHOOL

Apresentação de Slides

Modelagem Informacional

Prof. Dr. Wagner Perin
FUCAPE Business School

Integração de Dados e Arquitetura de Pipelines

O Paradigma ETL vs. ELT, dbt, Apache Airflow e APIs RESTful

Disciplina: Modelagem Informacional
Programa: Ciência de Dados para Negócios | **Carga Horária:** 4 h/a
Professor: Prof. Dr. Wagner Perin

Recapitulando: Onde estamos na Jornada de Dados?

- **Aulas 01-02:** Armazenamento colunar (Parquet) e *Predicate Pushdown*.
- **Aulas 03-04:** Modelagem relacional OLTP em 3FN com integridade transacional (ACID).
- **Aula 05:** Modelagem Multidimensional para relatórios analíticos (Star Schema).
- **Aula 06:** NoSQL e Persistência Poliglota (MongoDB, Neo4j, Redis).

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Dilema Crítico: Como unificar dados de silos operacionais e APIs externas em uma esteira analítica corporativa resiliente, sem derrubar a produção e sem gerar duplicações financeiras?

Hoje: Arquitetura ELT moderna com `dbt`, ingestão resiliente e orquestração idempotente no Airflow.

FUCAPE
BUSINESS SCHOOL

MÓDULO / SEÇÃO

Módulo 1: Abertura e Dor de Negócio

O Colapso Noturno da FinPay e as Falhas em Cascata

O Colapso do Pipeline Noturno na FinPay & OmniTrade

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Problema Investigado: A FinPay intermedia 250.000 transações diárias entre 50.000 lojistas. Às 02h00, um script Python monolítico legado em EC2 (8 GB RAM) extrai APIs bancárias, sofre bloqueio por HTTP 429, entra em OOM Killer e, após retry manual desastroso, duplica R$ 1,8 milhão no faturamento.



Painel de Monitoramento no Colapso Noturno da FinPay

As 4 Causas-Raiz do Colapso da FinPay

1. **Gargalo Computacional de Memória RAM:**
   - Script monólito acumulou milhões de linhas em DataFrames Pandas dentro de uma instância EC2 modesta. Resultado: `Out Of Memory (OOM) Killer` do Linux.
2. **Fragilidade de Rede e Falta de Governança de Tráfego:**
   - Consumo cego da API parceira ignorando *Rate Limits*. Ao receber `HTTP 429`, insistiu em laço rápido e sofreu bloqueio por 2 horas.
3. **Inexistência de Idempotência Computacional:**
   - Reexecução manual às 07h30 relendo dados parciais sem limpeza prévia. Duplicação de R$ 1,8 milhão em liquidações a lojistas.
4. **Acoplamento Severo e Descarte de Dados Brutos:**
   - Regras no meio da extração descartaram dados brutos. Impossível recalcular métricas históricas solicitadas pelo C-Level.

Por que Pipelines Tradicionais Quebram? As 4 Forças

- **1. Volumetria Dinâmica (*Volume Spikes*):**
  - Picos sazonais que superam a memória estática de servidores intermediários.
- **2. Mudança Silenciosa de Esquema (*Schema Drift*):**
  - Provedor altera campos no JSON da API sem aviso, quebrando schemas rígidos.
- **3. Instabilidade de Redes Distribuídas (*Network Flakiness*):**
  - Latências e quedas de conexão TLS que derrubam pipelines sem recuo inteligente.
- **4. Acoplamento entre Extração e Regras de Negócio:**
  - Alterar uma taxa exige reescrever a extração e reprocessar toda a esteira.

MÓDULO / SEÇÃO

FUCAPE
BUSINESS SCHOOL

Módulo 2: O Paradigma ETL vs. ELT Moderno

Da Escassez de Hardware On-Premises à Computação Elástica em Nuvem

A Morte do ETL Tradicional (Anos 1990/2000)

A Origem Histórica do ETL (*Extract, Transform, Load*):

- Servidores de banco de dados *on-premises* (Oracle, Teradata) tinham CPU e disco rígido extremamente caros.
- Criava-se um servidor intermediário (Pentaho, Informatica, scripts Python) para filtrar, limpar e resumir os dados antes de gravar no DW.

Os 3 Gargalos Fatais do Modelo Legado:

- **Ponto Único de Falha (SPOF):** O servidor intermediário satura CPU e memória com facilidade.
- **Perda Irreversível do Dado Bruto:** Registros descartados no filtro não podem ser recuperados para análises futuras.
- **Dependência Crônica de Engenharia:** Qualquer novo KPI exige alterar o script intermediário e reprocessar fontes originais.

A Revolução do ELT Moderno na Nuvem

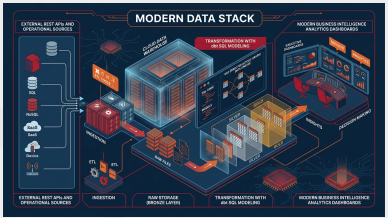
A Virada Tecnológica da Nuvem:

- Armazenamento de custo irrisório (AWS S3, Google Cloud Storage, Azure Blob).
- Motores analíticos MPP com **separação total entre armazenamento e computação elástica** (Snowflake, BigQuery, Databricks).

A Nova Dinâmica do ELT (*Extract, Load, Transform*):

- **Extract & Load:** Ingestão leve gravando o dado bruto exatamente como chegou (*Raw Data / Bronze Layer*).
- **Transform:** Toda a transformação e modelagem dimensional ocorre **dentro do Data Warehouse** via SQL e dbt.
- **Auditabilidade Total:** O dado bruto original nunca é destruído. Se uma regra de negócio mudar, o histórico é reprocessado em minutos.

Arquitetura Conceitual do Modern Data Stack

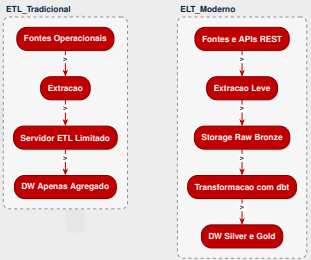


Arquitetura Conceitual do Modern Data Stack

INSIGHT EXECUTIVO

Insight Executivo: O Modern Data Stack desacopla a ingestão bruta da lógica de negócios, permitindo que Analytics Engineers transformem dados em escala com computação elástica sob demanda.

Fluxo Comparativo de Arquitetura: ETL vs. ELT



INSIGHT EXECUTIVO

Insight Executivo: No ELT, o dado bruto nunca é destruído. Se uma regra de negócio mudar, o histórico pode ser reprocessado instantaneamente dentro do DW.

Ingestão Resiliente de APIs RESTful: Rate Limit

O Mecanismo de Controle de Tráfego em APIs:

- Para proteger seus servidores contra sobrecarga (DoS), provedores impõem limites de requisições por intervalo de tempo.
- Quando o limite estoura, a API responde: `HTTP 429 Too Many Requests`.

Cabeçalhos HTTP Essenciais para Inspeção pelo Pipeline:

- `X-RateLimit-Limit`: Volume máximo de requisições permitidas na janela.
- `X-RateLimit-Remaining`: Quantidade de chamadas restantes antes do bloqueio.
- `X-RateLimit-Reset`: Timestamp UTC de quando a cota será renovada.
- `Retry-After`: Tempo exato em segundos que o cliente deve aguardar.

Regra de Ouro: *Pipelines profissionais jamais ignoram `HTTP 429` e nunca fazem retries imediatos em laço fechado.*

Algoritmo de Exponential Backoff com Jitter

O Erro do Retry Estático (`sleep` fixo):

- Se centenas de instâncias falharem no mesmo instante e todas aguardarem 2 segundos cravados, atacarão a API no mesmo milissegundo (*Thundering Herd Problem*).

A Solução Matemática: Recuo Exponencial com Jitter:

$$\Delta t = \min\left(t_{\max},\ t_{\text{base}} \times 2^{\text{tentativa}}\right) + \text{jitter}$$

- Onde t_{base} é o atraso inicial, $2^{\text{tentativa}}$ gera crescimento exponencial do tempo de espera e o **Jitter** ($\mathcal{U}(0, \text{jitter}_{\max})$) adiciona ruído aleatório.
- **Efeito Prático:** Dispersa as requisições concorrentes no tempo, garantindo restabelecimento suave sem derrubar o parceiro.

Ingestão Resiliente em Python com Backoff

Python – Código Fonte

```python
import time, random, requests

def extrair_pagina_resiliente(url, params, max_retries=5):
    tentativa = 0
    while tentativa < max_retries:
        resp = requests.get(url, params=params, timeout=10)
        if resp.status_code == 200:
            return resp.json()
        elif resp.status_code == 429:
            tentativa += 1
            retry_after = int(resp.headers.get("Retry-After", 2))
            jitter = random.uniform(0.1, 0.8)
            espera = (retry_after * (2 ** (tentativa - 1))) + jitter
            time.sleep(espera)
        else:
            resp.raise_for_status()
    raise TimeoutError(f"Falha de taxa excedida apos {max_retries} tentativas")
```

Prática de Engenharia: *O payload obtido é gravado diretamente em formato JSONL bruto no storage (Bronze Layer), sem descarte de colunas.*

MÓDULO / SEÇÃO

FUCAPE
BUSINESS SCHOOL

Módulo 3: Engenharia de Transformação com dbt

SQL Declarativo, Linhagem Automática e Governança de Dados

O que é o dbt e o Papel do Analytics Engineer?

- **O que o dbt NÃO faz:** Não extrai nem carrega dados de fora.
- **O que o dbt FAZ:** Orquestra e executa todas as transformações **dentro do Data Warehouse existente**.
- **O Paradigma do SQL Declarativo:**
  - O engenheiro escreve apenas instruções `SELECT`.
  - O compilador do dbt gera autonomamente o DDL/DML correspondente (`CREATE TABLE AS SELECT`, `CREATE VIEW`, `MERGE INTO`).
- **Engenharia de Software Aplicada a Dados:**
  - Versionamento estrito com Git.
  - Testes automatizados de qualidade antes da publicação em produção.
  - Documentação viva e linhagem de dependências calculada em tempo de compilação.

A Macro ref() e a DAG de Linhagem Automática

O Fim do Acoplamento com Nomes Físicos de Tabelas:

- Em vez de codificar `FROM prod_dw.raw.tb_transacoes`, referencia-se o modelo lógico:

`FROM {{ ref('stg_finpay_transacoes') }}`.

Benefícios Arquiteturais Cardeais:

1. **Linhagem Dinâmica (DAG):** O dbt inspeciona todas as macros `{{ ref() }}`, deduz a árvore de dependências e paraleliza a execução automaticamente.
2. **Independência de Ambiente:** O mesmo código roda em *Sandbox/Dev* ou *Produção* apenas alternando o target do `profiles.yml`.

Organização em Camadas no dbt:

- **Staging (`stg_`):** Limpeza primária, renomeação de colunas e tipagem (*casting*).
- **Intermediate (`int_`):** Junções e consolidação de entidades de negócio.
- **Marts (`fct_` e `dim_`):** Modelos dimensionais de negócio prontos para BI e ML.

Modelos dbt na Prática: Camada Staging

SQL – Código Fonte

```sql
-- models/staging/stg_finpay_transacoes.sql
{{ config(materialized='view') }}
SELECT
    CAST(transacao_id AS VARCHAR(64)) AS id_transacao,
    CAST(lojista_id AS VARCHAR(32)) AS id_lojista,
    CAST(valor_centavos / 100.0 AS NUMERIC(15, 2)) AS vlr_transacao_brl,
    UPPER(TRIM(status)) AS des_status_transacao,
    CAST(data_hora_utc AS TIMESTAMP) AS dth_transacao_utc
FROM {{ source('finpay_raw', 'tb_transacoes_raw') }}
WHERE transacao_id IS NOT NULL;
```

Camada Staging: *Padroniza nomes, tipos e filtros de saneamento elementares sem alterar a granularidade natural da fonte de dados.*

Modelos dbt na Prática: Camada Marts Incremental

SQL – Código Fonte

```sql
-- models/marts/fct_liquidacoes_diarias.sql
{{ config(materialized='incremental', unique_key='id_transacao') }}
SELECT
    id_transacao, id_lojista, vlr_transacao_brl,
    ROUND(vlr_transacao_brl * 0.025, 2) AS vlr_taxa_mdr,
    ROUND(vlr_transacao_brl * 0.975, 2) AS vlr_liquido_lojista
FROM {{ ref('stg_finpay_transacoes') }}
WHERE des_status_transacao = 'APROVADO'
{% if is_incremental() %}
  AND dth_transacao_utc > (SELECT MAX(dth_transacao_utc) FROM {{ this }})
{% endif %};
```

Materialização Incremental: *Processa apenas os registros novos ou modificados da janela recente, reduzindo custos de computação no DW.*

Testes Declarativos de Qualidade no dbt

Contratos de Dados e Integridade Referencial no schema.yml

Controle Bloqueante: Se `dbt test` falhar no CI/CD, o pipeline é abortado antes de impactar o DW.

YAML – Código Fonte

```yaml
version: 2
models:
  - name: stg_finpay_transacoes
    columns:
      - name: id_transacao
        tests: [unique, not_null]
      - name: des_status_transacao
        tests:
          - not_null
          - accepted_values: {values: ['APROVADO', 'RECUSADO']}
  - name: fct_liquidacoes_diarias
    columns:
      - name: id_transacao
        tests: [unique, not_null]
      - name: id_lojista
        tests: [relationships: {to: ref('dim_lojistas'), field: id_lojista}]
```

MÓDULO / SEÇÃO

FUCAPE
BUSINESS SCHOOL

Módulo 4: Orquestração e Idempotência no Airflow

Workflows-as-Code e Eliminação Definitiva de Duplicatas

Apache Airflow: O Maestro da Engenharia de Dados

- **Por que ferramentas como o cron tradicional falham?**
  - Sem controle visual de dependências complexas entre tarefas.
  - Sem retries automáticos com intervalo configurável por falha.
  - Sem telemetria, logs centralizados ou alertas nativos.
- **Fundamentos do Apache Airflow:**
  - **DAG (*Directed Acyclic Graph*):** Grafo direcionado e sem loops infinitos.
  - **Operadores:** Unidades de execução (`PythonOperator`, `BashOperator`).
  - **Janela Lógica (`{{ ds }}`):** Execuções parametrizadas pelo intervalo do dado.
  - **Workflows-as-Code:** Toda a esteira é versionada em Python puro.

Painel de Orquestração DAG no Apache Airflow



Painel de Orquestração DAG no Apache Airflow

INSIGHT EXECUTIVO

Insight Executivo: A telemetria visual do Airflow rastreia a linhagem temporal e o status de execução de cada nó do pipeline com monitoramento em tempo real.

O Princípio Sagrado da IDEMPOTÊNCIA

"Uma operação é dita IDEMPOTENTE se executá-la uma única vez ou cinquenta vezes com os mesmos parâmetros resulta rigorosamente no mesmo estado final: $f(f(x)) = f(x)$."

Por que a Idempotência é Inegociável?

- Redes falham, APIs caem e servidores sofrem reinicializações.
- Falhas transitórias exigem retries automáticos ou manuais.
- Um pipeline sem idempotência duplica registros e distorce faturamentos (caso dos R$ 1,8 milhão da FinPay).
- **Regra de Engenharia:** Todo pipeline de dados deve ser projetado para que reexecuções parciais ou totais jamais corrompam o estado analítico.

Estratégias de Idempotência: Delete-and-Insert e Upsert

Garantia Matemática: Zero Duplicações em Qualquer Reexecução

1. Delete-and-Insert Particionado por Data Lógica ({{ ds }}):

SQL – Código Fonte

```sql
DELETE FROM dw.tb_fato WHERE dat_particao = '{{ ds }}';
INSERT INTO dw.tb_fato SELECT ... WHERE dat_transacao = '{{ ds }}';
```

2. Merge / Upsert via Chave Primária de Negócio:

SQL – Código Fonte

```sql
MERGE INTO dw.tb_fato T USING stg S ON T.id = S.id
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...;
```

DAG Idempotente no Apache Airflow (Código Fonte)

Python – Código Fonte

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {'owner': 'dados', 'retries': 3, 'retry_delay': timedelta(minutes=5)}

with DAG('finpay_elt_diario', default_args=default_args,
         schedule_interval='0 3 * * *', start_date=datetime(2026, 1, 1),
         catchup=False, max_active_runs=1) as dag:

    t1 = BashOperator(task_id='extrair_api',
                      bash_command='python /scripts/ingestao.py --data {{ ds }}')
    t2 = BashOperator(task_id='carga_raw_idempotente',
                      bash_command='python /scripts/carga_raw.py --data {{ ds }}')
    t3 = BashOperator(task_id='dbt_run', bash_command='dbt run')
    t4 = BashOperator(task_id='dbt_test', bash_command='dbt test')

    t1 >> t2 >> t3 >> t4
```

Desafio Prático de Código (PBL): O Pipeline FinPay

DESAFIO PRÁTICO DE CÓDIGO (PBL)

Desafio Prático de Código (PBL): Assumindo o papel de Senior Analytics Engineers, as equipes devem reestruturar a esteira de dados da FinPay substituindo o monólito legado por uma arquitetura ELT moderna e tolerante a falhas.

Roteiro de Implementação em 3 Tarefas:

- **Tarefa 1 (Ingestão Resiliente Python):** Paginação por cursor, tratamento de HTTP 429 com Exponential Backoff e persistência JSONL (*Bronze Layer*).
- **Tarefa 2 (Modelagem e Qualidade dbt):** Modelos `stg_` e `fct_` incremental com testes bloqueantes no `schema.yml`.
- **Tarefa 3 (Orquestração Airflow):** DAG com dependências estritas e estratégia *Delete-and-Insert* por `{{ ds }}`.

Matriz Comparativa Definitiva: ETL vs. ELT

Dimensão de Comparação	ETL Tradicional (Legado / On-Prem)	ELT Moderno (Cloud-Native MPP)
Onde ocorre a transformação?	Servidor intermediário (Pentaho, EC2).	Dentro do próprio Cloud DW (Snowflake).
Momento da Carga (*Load*)	No final, após regras de negócio.	No início, logo após extração (*Raw Data*).
Preservação de Dados Brutos	Não. Descartados na transformação.	Sim (100%). Imutáveis na camada Bronze.
Escalabilidade Computacional	Vertical e limitada por CPU/RAM.	Horizontal e elástica sob demanda.
Linguagem Principal	Código imperativo proprietário / scripts.	SQL declarativo modular com dbt.
Tratamento de Schema Drift	Altamente frágil a novos campos.	Resiliente (armazenamento semiestruturado).
Tempo para Novas Métricas	Semanas de desenvolvimento.	Horas (novo modelo SQL sobre o Raw).
Perfil Profissional	Engenheiro de ETL especializado.	*Analytics Engineer* com SQL e visão negocial.

Reflexão Analítica & Dilema de Negócio

REFLEXÃO ANALÍTICA & DILEMA DE NEGÓCIO

Trade-off Estratégico: A FinPay enfrenta a decisão entre manter a infraestrutura legada EC2 (custo fixo de R$ 3.500/mês, mas com risco de multas de R$ 1,8 milhão) ou migrar para Cloud Data Warehouse com ELT (custo sob demanda de R$ 10.000/mês com 99,99% de disponibilidade).

Desafio Corporativo & Questões de Impacto para a Turma:

- Como quantificar o custo oculto da perda de confiança de lojistas versus a fatura de computação em nuvem?
- Por que a flexibilidade de reprocessar o dado bruto histórico é considerada um seguro estratégico para a empresa?
- Como a cultura de testes automatizados no dbt protege o time contra erros catastróficos em fechamentos contábeis?

Síntese da Aula e Teaser para a Avaliação A1

As 3 Lições Fundamentais da Aula 07:

1. **Separação de Preocupações:** Extração leve para garantir resiliência e integridade bruta; transformação delegada ao poder elástico do DW.
2. **Qualidade é Código:** O dbt profissionalizou o SQL com controle de versão, testes declarativos de schema e linhagem via `{{ ref() }}`.
3. **Idempotência é Inegociável:** Qualquer tarefa deve poder ser reexecutada sem risco de duplicação financeira ($f(f(x)) = f(x)$).

Atenção para a Próxima Aula (Aula 08):

- **Avaliação Individual Presencial A1:** Cobrindo Unidades 1, 2 e 3 (Formatos Físicos, Normalização 3FN, Star Schema, NoSQL e Arquitetura ELT).
- **Entrega Oficial do Trabalho Prático T1:** Submissão dos scripts e diagramas até o início da aula.